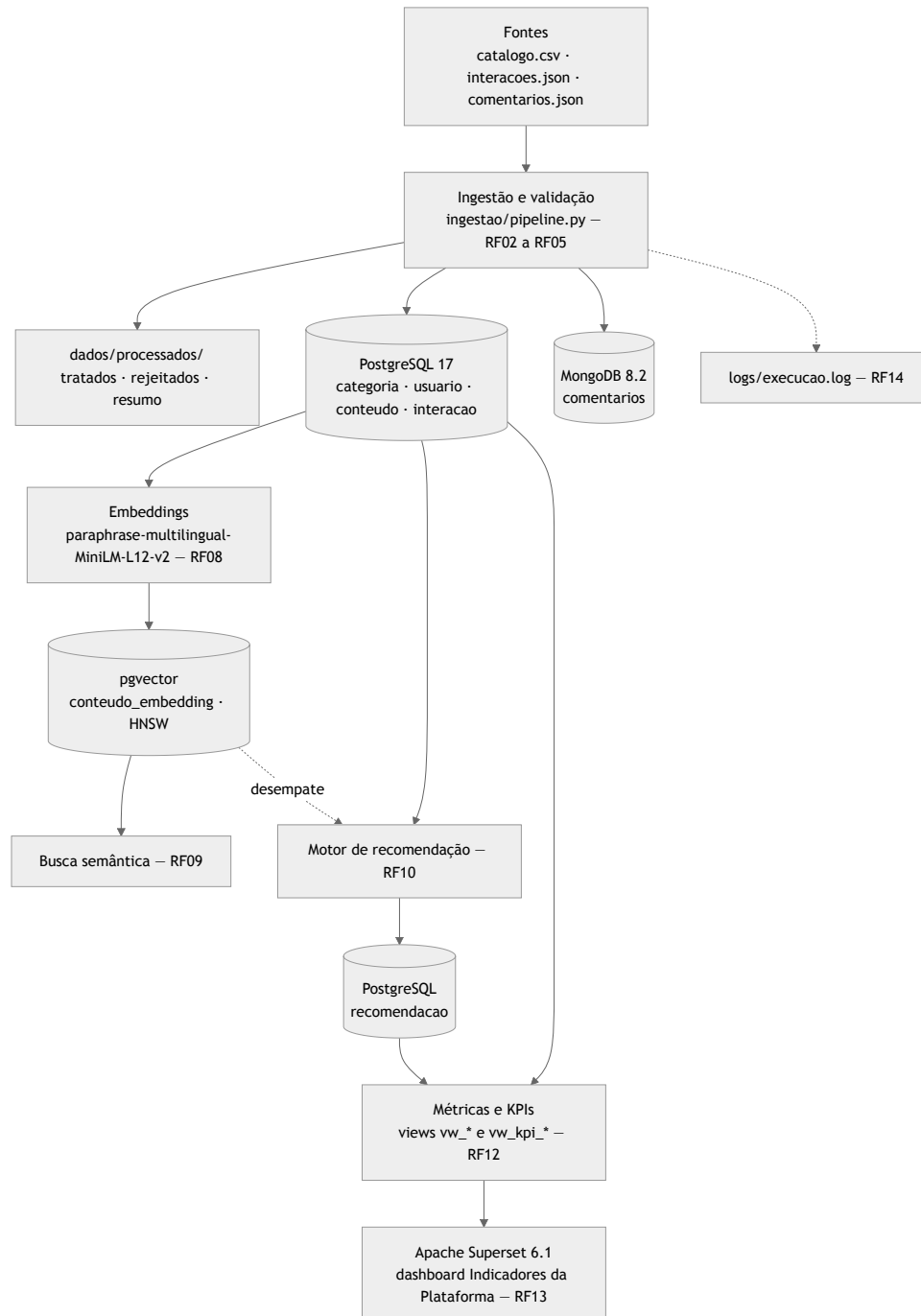


Arquitetura da solução

Desafio Prático 1 — Pipeline de Recomendação e Dashboard de Conteúdos Educacionais · Vynicius Yuji Mogami, Victor Griggi Moreira Regis da Silva, Kevin da Silva Medeiros

1. Visão geral

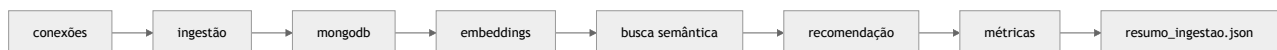
A solução segue a arquitetura mínima do enunciado (seção 8): fontes em CSV/JSON → ingestão e validação → PostgreSQL (estruturado), MongoDB (semiestruturado) e pgvector (vetorial) → busca semântica e motor de recomendação → Apache Superset. Tudo roda em contêineres Docker orquestrados por um único `docker-compose.yml`; o pipeline é executado por `python -m src.main` dentro do contêiner `app`.



2. Componentes e tecnologias

Etapa	Tecnologia	Onde
Fontes	CSV e JSON fictícios	dados/brutos/ (nunca alterados)
Ingestão	Python 3.12, biblioteca padrão	ingestao/pipeline.py
Dados estruturados	PostgreSQL 17 (imagem pgvector/pgvector:pg17)	sql/criar_banco.sql
Dados semiestruturados	MongoDB 8.2, pymongo	mongodb/comentarios.py , mongodb/consultas.js
Dados vetoriais	pgvector 0.8, extensão do mesmo PostgreSQL; índice HNSW por cosseno	conteudo_embedding
Embeddings	sentence-transformers, modelo multilíngue de 384 dimensões, CPU	recomendacao/embeddings.py
Busca semântica e recomendação	Python + SQL (pgvector)	recomendacao/
Métricas e KPIs	views SQL + src/metricas.py	sql/criar_banco.sql , documentacao/kpis.md
Apresentação	Apache Superset 6.1 (imagem oficial + driver psycopg2)	dashboard/evidencias/
Orquestração	Docker Compose: postgres , mongo , superset-init , superset , app	docker-compose.yml , docker/
Configuração	YAML para parâmetros; .env (fora do git) para senhas e portas	config.yaml , .env.example

3. Fluxo de execução do pipeline

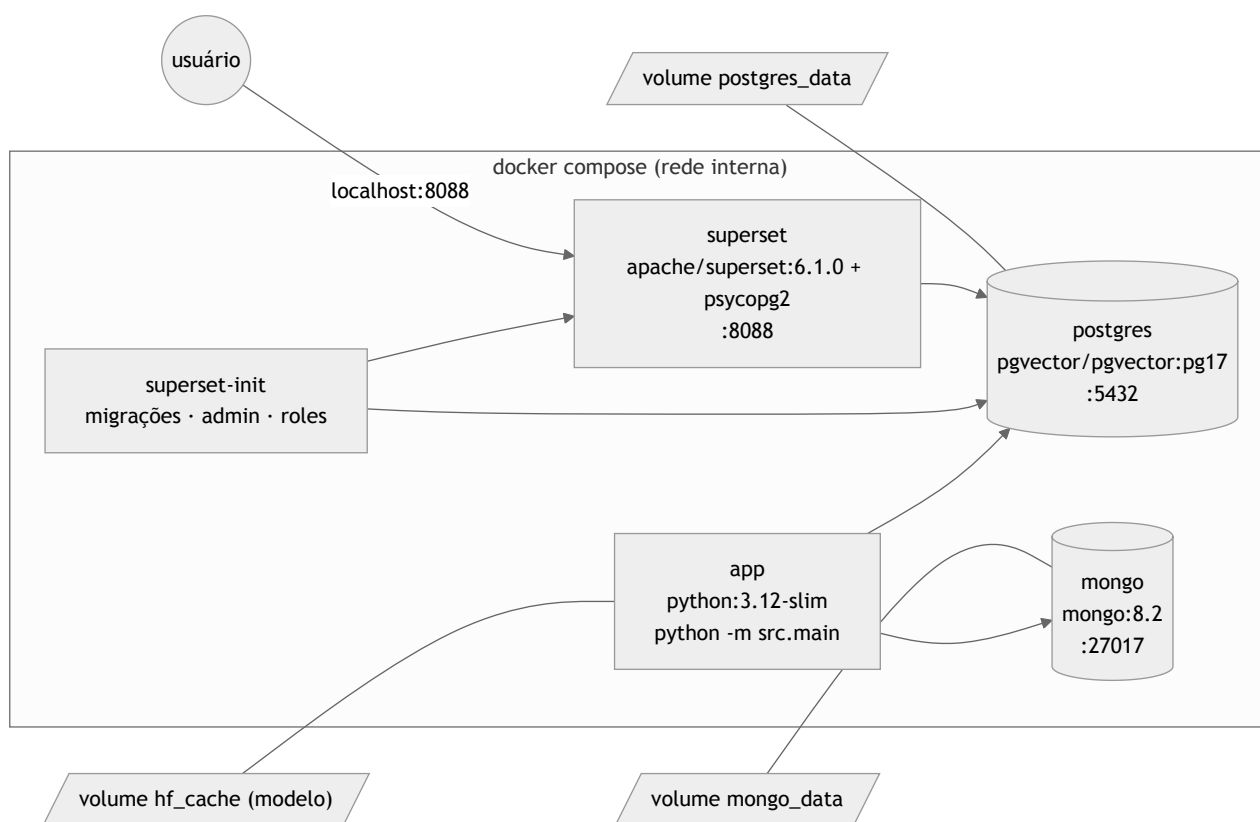


Etapa	RF	O que faz	Saída
conexões	RF01, RF14	abre PostgreSQL e MongoDB; falha de conexão interrompe com erro claro	—
ingestão	RF02–RF06	lê, classifica (válido/inválido/incompleto/duplicado), padroniza, grava processados e carrega o PostgreSQL em uma transação; reconcilia o que deixou de ser válido	processados, rejeitados.json , tabelas
mongodb	RF07	upsert dos comentários válidos com categoria/título/tipo desnormalizados; remove cargas antigas; exercita as consultas exigidas	coleção comentarios
embeddings	RF08	gera vetores só para conteúdos sem embedding do modelo configurado	conteudo_embedding
busca semântica	RF09	três consultas de demonstração em linguagem natural	busca_semantica.json
recomendação	RF10, RF11	$Pontuação = ((Ivis + Icur)/2) \times 100 \times Iconc$; top 10 por usuário, negativos descartados	recomendacao , recomendacoes.json

métricas	RF12	garante as views, registra a execução, calcula e grava os indicadores	views, execucao_pipeline, kpis.json
----------	------	---	---

Cada etapa é cronometrada e registrada em `logs/execucao.log`; o `resumo_ingestao.json` consolida contagens, tempos e o status da execução (sucesso ou falha).

4. Infraestrutura Docker



- `sql/criar_banco.sql` é aplicado automaticamente na criação do volume do PostgreSQL (tabelas, índices, views) e reaplicado pelo pipeline a cada execução (idempotente).
- O Superset usa um banco de metadados próprio (`superset`) no mesmo servidor PostgreSQL, criado no `init`.
- O modelo de embeddings (~470 MB) é baixado uma vez e cacheado em volume.
- Portas do host são configuráveis no `.env` (`POSTGRES_HOST_PORT` , `MONGO_HOST_PORT` , `SUPERSET_HOST_PORT`) para evitar conflitos.
- Reprodução completa: `cp .env.example .env && docker compose up -d && docker compose run --rm app`.

5. Práticas de DataOps (seção 9)

Prática	Como
Código organizado em diretórios	estrutura da seção 11 do enunciado; orquestrador em <code>src/</code>
Dependências registradas	<code>requirements.txt</code> com versões fixas; imagens Docker com tags fixas
Parâmetros de conexão separados do código	<code>config.yaml</code> + <code>.env</code> (ignorado pelo git)
Registro das etapas executadas	<code>logs/execucao.log</code> e tabela <code>execucao_pipeline</code>
Tratamento básico de erros	falhas de conexão, de embedding e de persistência registradas; código de saída 1 e <code>status: falha</code> no resumo

Reprodutibilidade	README com instruções; execuções repetidas produzem o mesmo estado (idempotência e reconciliação)
Controle de versão	Git com <code>.gitignore</code> e <code>.gitattributes</code>
Testes	<code>tests/</code> : validação com registros sujos, fórmula e classificação da recomendação